Práctica 2

Comunicaciones en red

Y

Concurrencia

Axel Sargenti
48767861H

Enlace al Github: https://github.com/axelsargenti-ua/Practica-2.git

Ejercicio 1: “COMUNICACIONES EN RED”

Implementar una aplicación cliente-servidor mediante la utilización de sockets cuya funcionalidad será la transmisión de un archivo desde el servidor al cliente. El ejercicio constará de dos partes (procesos):

El servidor. Proceso que estará esperando a que le lleguen conexiones desde el cliente y, una vez establecida la conexión, transferirá al cliente el contenido de un archivo denominado “Google.html”. El contenido del archivo será el código fuente de la página web de Google.

El cliente. Proceso que establecerá la conexión con el servidor, recibirá el contenido del archivo y lo mostrará por pantalla. El cliente se ejecutará en la máquina local y se invocará con la orden: Cliente IP_Servidor El servidor se lanzará con la orden Servidor y debe estar en todo momento escuchando por el puerto 9999, preparado para aceptar conexiones. Tras una petición de conexión por parte de un cliente, debe crear un hijo que será el encargado de realizar todas las operaciones necesarias para la transferencia del archivo.

Resolución:

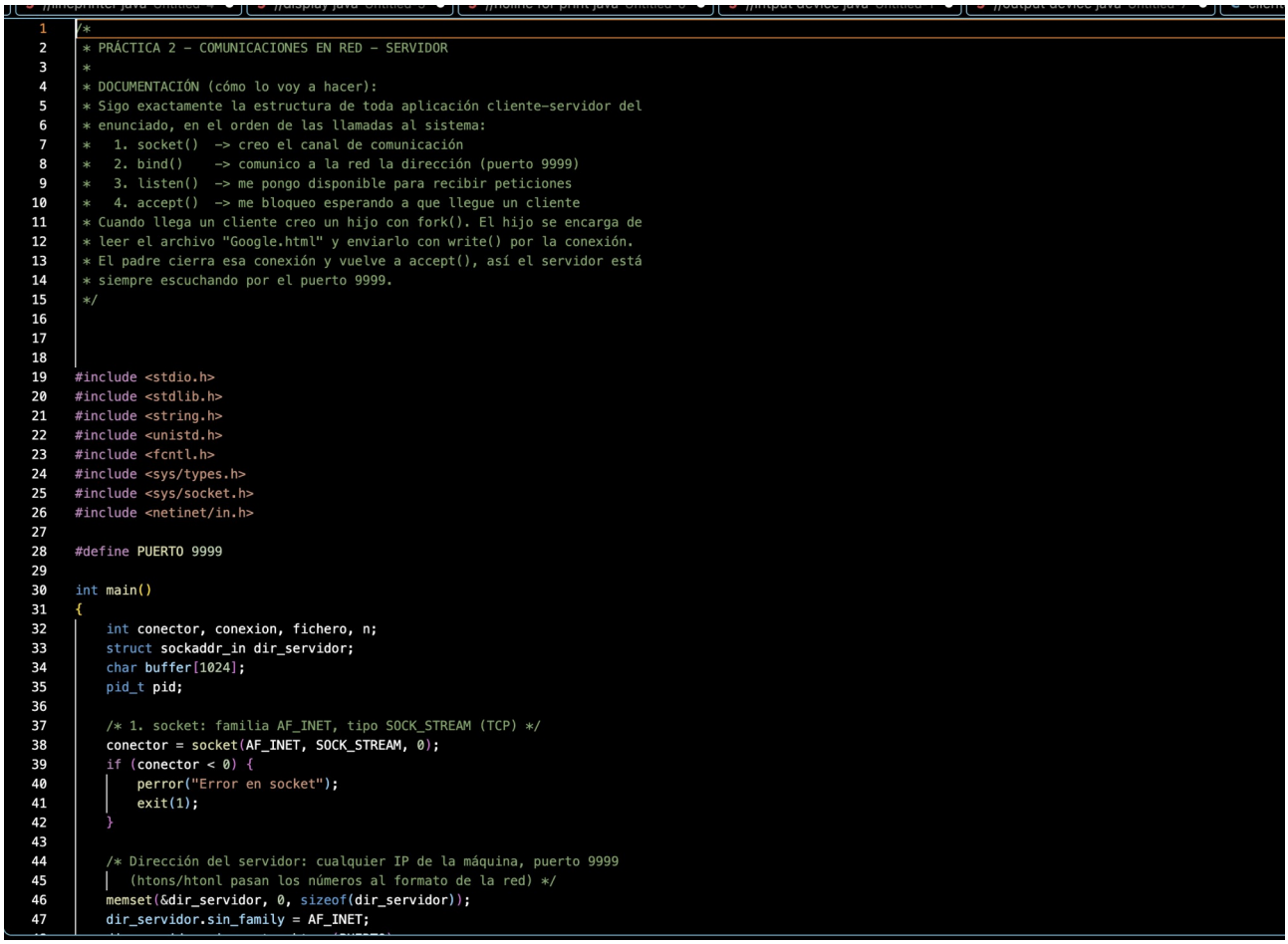
El enunciado pide transmitir el archivo Google.html del servidor al cliente. Para ello sigo la estructura de toda aplicación cliente-servidor:

Servidor	Cliente
socket() crea el canal	socket() crea el canal
bind() comunica la dirección (puerto 9999)	
listen() queda disponible para peticiones	
accept() se bloquea esperando clientes	connect() pide la conexión
read()/write()	read()/write()
close()	close()

Pasos:

1. Servidor: crea el socket, lo asocia al puerto 9999 con bind, llama a listen y entra en un bucle con accept. Cuando llega un cliente crea un hijo con fork(); el hijo lee Google.html y lo envía con write; el padre cierra la conexión y vuelve a accept, de forma que siempre está escuchando.
2. Cliente: recibe la IP del servidor como argumento (Cliente IP_Servidor), la convierte con inet_addr, hace connect y va leyendo con read lo que llega, mostrándolo por pantalla con write, hasta que el servidor cierra la conexión.
3. Implemento las funciones socket, bind, listen, accept, connect, inet_addr, htons, htonl y las estructuras sockaddr_in / sockaddr.

Código:



```c
/*
 * PRÁCTICA 2 - COMUNICACIONES EN RED - SERVIDOR
 *
 * DOCUMENTACIÓN (cómo lo voy a hacer):
 * Sigo exactamente la estructura de toda aplicación cliente-servidor del
 * enunciado, en el orden de las llamadas al sistema:
 *   1. socket()  -> creo el canal de comunicación
 *   2. bind()    -> comunico a la red la dirección (puerto 9999)
 *   3. listen()  -> me pongo disponible para recibir peticiones
 *   4. accept()  -> me bloqueo esperando a que llegue un cliente
 * Cuando llega un cliente creo un hijo con fork(). El hijo se encarga de
 * leer el archivo "Google.html" y enviarlo con write() por la conexión.
 * El padre cierra esa conexión y vuelve a accept(), así el servidor está
 * siempre escuchando por el puerto 9999.
 */



#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>

#define PUERTO 9999

int main()
{
    int conector, conexion, fichero, n;
    struct sockaddr_in dir_servidor;
    char buffer[1024];
    pid_t pid;

    /* 1. socket: familia AF_INET, tipo SOCK_STREAM (TCP) */
    conector = socket(AF_INET, SOCK_STREAM, 0);
    if (conector < 0) {
        perror("Error en socket");
        exit(1);
    }

    /* Dirección del servidor: cualquier IP de la máquina, puerto 9999
       (htons/htonl pasan los números al formato de la red) */
    memset(&dir_servidor, 0, sizeof(dir_servidor));
    dir_servidor.sin_family = AF_INET;
```

```c
    memset(&dir_servidor, 0, sizeof(dir_servidor));
    dir_servidor.sin_family = AF_INET;
    dir_servidor.sin_port = htons(PUERTO);
    dir_servidor.sin_addr.s_addr = htonl(INADDR_ANY);

    /* 2. bind */
    if (bind(conector, (struct sockaddr *)&dir_servidor, sizeof(dir_servidor)) < 0) {
        perror("Error en bind");
        exit(1);
    }

    /* 3. listen: cola de 5 peticiones pendientes */
    if (listen(conector, 5) < 0) {
        perror("Error en listen");
        exit(1);
    }
    printf("Servidor escuchando en el puerto %d\n", PUERTO);

    while (1) {
        /* 4. accept: espera a que llegue un cliente */
        conexion = accept(conector, NULL, NULL);
        if (conexion < 0) {
            perror("Error en accept");
            continue;
        }

        pid = fork();
        if (pid == 0) {
            /* HIJO: transfiere el archivo al cliente */
            close(conector);
            fichero = open("Google.html", O_RDONLY);
            if (fichero < 0) {
                perror("Error al abrir Google.html");
                exit(1);
            }
            while ((n = read(fichero, buffer, sizeof(buffer))) > 0)
                write(conexion, buffer, n);
            close(fichero);
            close(conexion);
            exit(0);
        }
        /* PADRE: no usa esta conexión, vuelve a esperar clientes */
        close(conexion);
    }
}
```

```c
/*
 * PRÁCTICA 2 - COMUNICACIONES EN RED - CLIENTE
 *
 * DOCUMENTACIÓN (cómo lo voy a hacer):
 * Sigo la estructura de toda aplicación cliente-servidor del enunciado:
 *   1. socket()   -> creo el canal de comunicación
 *   2. connect()  -> pido la conexión al servidor (su IP la recibo como
 *                    argumento y la convierto con inet_addr(); puerto 9999)
 *   3. read()     -> leo lo que envía el servidor, hasta que cierra la
 *                    conexión, y lo muestro por pantalla con write()
 *   4. close()    -> cierro el canal
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define PUERTO 9999

int main(int argc, char *argv[])
{
    int conector, n;
    struct sockaddr_in dir_servidor;
    char buffer[1024];

    if (argc != 2) {
        printf("Uso: Cliente IP_Servidor\n");
        exit(1);
    }

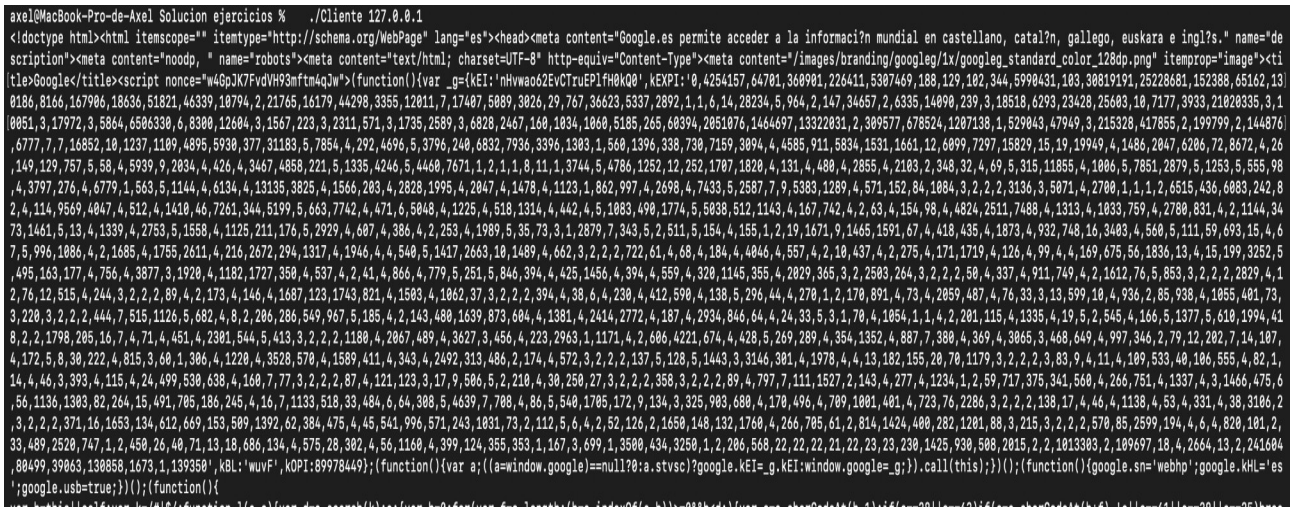
    /* 1. socket */
    conector = socket(AF_INET, SOCK_STREAM, 0);
    if (conector < 0) {
        perror("Error en socket");
        exit(1);
    }

    /* Dirección del servidor: IP recibida como argumento y puerto 9999 */
    memset(&dir_servidor, 0, sizeof(dir_servidor));
    dir_servidor.sin_family = AF_INET;
    dir_servidor.sin_port = htons(PUERTO);
    dir_servidor.sin_addr.s_addr = inet_addr(argv[1]);

```

```
45      dir_servidor.sin_port = htons(PUERTO);
46      dir_servidor.sin_addr.s_addr = inet_addr(argv[1]);
47
48      /* 2. connect */
49      if (connect(conector, (struct sockaddr *)&dir_servidor, sizeof(dir_servidor)) < 0) {
50          perror("Error en connect");
51          exit(1);
52      }
53
54      /* 3. read del socket y write por pantalla (descriptor 1) */
55      while ((n = read(conector, buffer, sizeof(buffer))) > 0)
56          write(1, buffer, n);
57
58      /* 4. close */
59      close(conector);
60      return 0;
61  }
```

Ejecución



EJERCICIO 2: “CONCURRENCIA”

Implementar mediante semáforos el problema del productor consumidor con buffer limitado estudiado en clase de teoría.

Resolución:

Es el problema del productor-consumidor con buffer limitado de teoría: dos procesos y un buffer de tamaño N. Hay que resolver dos cosas, y uso un semáforo para cada una .

Problema	Semáforo	Valor inicial	En teoría
Exclusión mutua (sección crítica)	mutex	1	S
El consumidor no puede coger si está vacío	llenos	0	C
El productor no puede dejar si está lleno	vacios	N	P

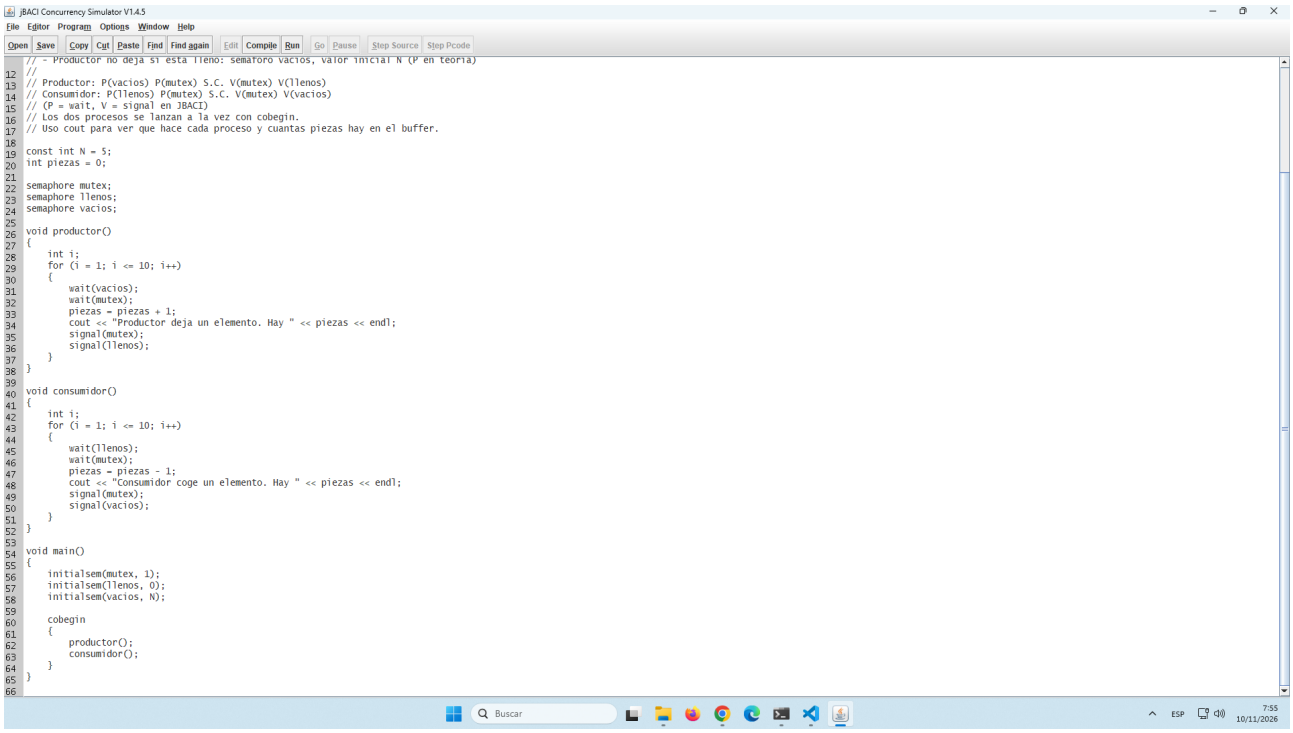
Esquema de cada proceso (P = wait, V = signal):

Productor:	Consumidor:
P(vacios)	P(llenos)
P(mutex)	P(mutex)
sección crítica	sección crítica
V(mutex)	V(mutex)
V(llenos)	V(vacios)

- El wait de sincronización va antes que el de mutex: si fuera al revés, un proceso podría quedarse bloqueado dentro de la sección crítica con mutex cogido y el otro nunca podría liberarlo (interbloqueo).
- Los dos procesos se lanzan a la vez con cobegin.

- Para ver qué ocurre uso cout: cada proceso escribe lo que hace y cuántos elementos hay en el buffer.

Código



```
// - Productor no deja si esta lleno: semaforo vacios, valor inicial N (P en teoria)
//
// Productor: P(vacios) P(mutex) S.C. V(mutex) V(llenos)
// Consumidor: P(llenos) P(mutex) S.C. V(mutex) V(vacios)
// (P = wait, V = signal en JBACI)
// Los dos procesos se lanzan a la vez con cobegin.
// Uso cout para ver que hace cada proceso y cuantas piezas hay en el buffer.

const int N = 5;
int piezas = 0;

semaphore mutex;
semaphore llenos;
semaphore vacios;

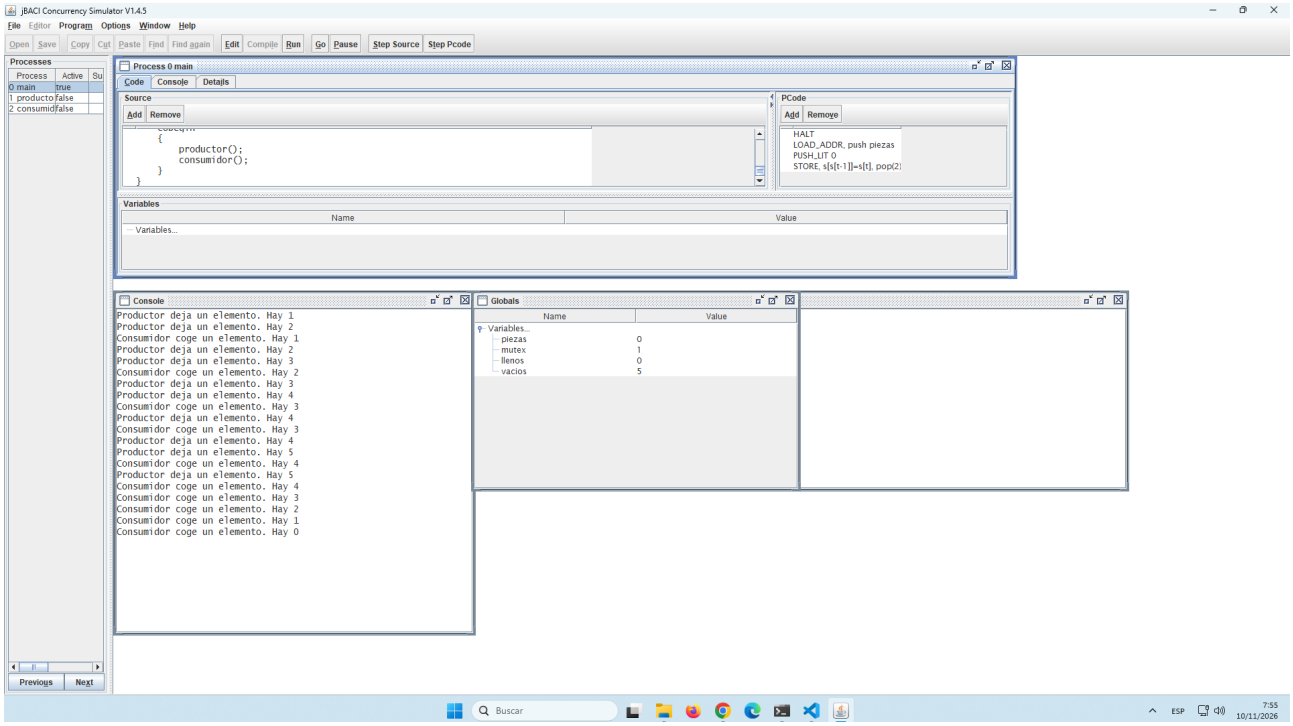
void productor()
{
    int i;
    for (i = 1; i <= 10; i++)
    {
        wait(vacios);
        wait(mutex);
        piezas = piezas + 1;
        cout << "Productor deja un elemento. Hay " << piezas << endl;
        signal(mutex);
        signal(llenos);
    }
}

void consumidor()
{
    int i;
    for (i = 1; i <= 10; i++)
    {
        wait(llenos);
        wait(mutex);
        piezas = piezas - 1;
        cout << "Consumidor coge un elemento. Hay " << piezas << endl;
        signal(mutex);
        signal(vacios);
    }
}

void main()
{
    initialsem(mutex, 1);
    initialsem(llenos, 0);
    initialsem(vacios, N);

    cobegin
    {
        productor();
        consumidor();
    }
}
```

Ejecución



```
Productor deja un elemento. Hay 1
Productor deja un elemento. Hay 2
Consumidor coge un elemento. Hay 1
Productor deja un elemento. Hay 2
Productor deja un elemento. Hay 3
Consumidor coge un elemento. Hay 2
Productor deja un elemento. Hay 3
Productor deja un elemento. Hay 4
Consumidor coge un elemento. Hay 3
Productor deja un elemento. Hay 4
Consumidor coge un elemento. Hay 3
Productor deja un elemento. Hay 4
Productor deja un elemento. Hay 5
Consumidor coge un elemento. Hay 4
Productor deja un elemento. Hay 5
Consumidor coge un elemento. Hay 4
Consumidor coge un elemento. Hay 3
Consumidor coge un elemento. Hay 2
Consumidor coge un elemento. Hay 1
Consumidor coge un elemento. Hay 0
```

El número de elementos nunca baja de 0 ni supera 5.